

CSC-648-01-SU26

Project: Student Center

Team #4: BigBacks

Version History

Milestone 4 Version 1	7/22/2026

Team Members

Wen Chien Yen	Team Lead / Scrum Master	wyen@sfsu.edu
Oscar Wang	Frontend Lead / Technical Writer	owang1@sfsu.edu
Banghao Yuan	Software Architect	Byuan1@sfsu.edu
Sean Wang	Database Administrator	swang29@sfsu.edu
Enoch Lin	Github Master	elin5@sfsu.edu
Kashif Zada	Backend Lead	kzada@sfsu.edu

Revision History

Date	Member	Task	Details
7/27/2025	Oscar Wang	Frontend	Navbar will now always send the user to the site's homepage when clicking on the logo and name, since there is a dedicated "portal" link in the profile button. Also a slight grammar fix to the homepage. Also fixed a sizing issue with the notifications button for students.
7/27/2026	Kashif Zada	Code Review	Completed external code review of Team 1 (TerrorByte-01), identifying a duplicated income/expense calculation bug across RecommendationService, TrueCostCalculatorService, and SubscriptionService missing a null-amount filter present in FinanceService; added code screenshots as evidence.

7/27/2026	Sean Wang	Documentation	Added Usability Test Plan survey result analysis based on responses we have received so far.
7/27/2026	Enoch Lin	Documentation	Added Localization Testing documentation
7/27/2026	Banghao Yuan	Frontend	On mobile screens, page sections now use single-column layouts, and prevent images from overflowing the viewport.
7/25/2026	Wen Chien Yen	Deployment / Prototype Verification	Verified that the deployed prototype remained accessible through the public EC2 URL, confirmed that the backend service stayed running through PM2, and checked that the prototype testing workflow matched the credentials README and main README before submission.
7/25/2026	Wen Chien Yen	Usability Testing / Team Contributions	Helped organize the usability testing direction by confirming that login and registration should be excluded, recommending student-facing test tasks such as Course Search, Cart, Schedule, Notifications, and Degree Progress, and prepared the Team Contributions section based on M4 work history and

			assigned responsibilities.
7/25/2026	Sean Wang	Documentation	Finalized final list of usability test tasks. Wrote Test Setup and User Profile management section. Made Google Form for usability test.
7/25/2026	Enoch Lin	Github/CI	Added a CI workflow that runs frontend lint and build checks and backend tests automatically for every PR.
7/25/2026	Banghao Yuan	Backend	Add welcome message display for new signup User
7/25/2026	Sean Wang	Documentation	Set up documentation template for Usability Test Plan section.
7/25/2026	Sean Wang	Documentation	Finished Security Self Check section & added images of our code for each security check.
7/25/2026	Sean Wang	Documentation	Finished AI Feature Testing and Ethics Review section (our application does not use AI features so I wrote an explanation)
7/24/2026	Banghao Yuan	Documentation	Create each Task for Usability test.
7/24/2026	Sean Wang	Documentation	Finished Non Functional Requirements Status section.
7/24/2026	Banghao Yuan	Code review/Backend	Checking the repo, fix the issue that

			reviewer point out on the comment, and fix it and add our code review process.
7/24/2026	Banghao Yuan	Backend	Add the decision logic, if register with email end by BBedu, then skip the invite process. (for Usability test)
7/24/2026	Banghao Yuan	QA Test	Add the Reliability, Security, Compatibility, Robustness and Performance section of QA Test.
7/24/2026	Sean Wang	Documentation	Finished Product Summary section, refined list of functional requirements, rewrote list of unique features.
7/23/2026	Oscar Wang	Frontend	Slight changes to new frontend branch, foundation for new profile icon for logged in students.
7/23/26	Banghao Yuan	Frontend and Backend	Enhanced admin course and degree requirement management by adding course pagination, bulk import, Major Elective and GE Area support, improved Degree Requirement pages, and automatic Class Section date alignment with semester dates.

7/21/2026	Wen Chien Yen	Team Coordination / Testing Planning / Submission Support	Reviewed the Milestone 4 README requirements, coordinated the team split between Engineering Group and Testing/Documentation Group, assigned section ownership for M4v1, helped define the five usability test functions, reviewed prototype testing flow, and prepared communication for instructor submission and follow-up.
-----------	---------------	---	--

Table of Contents

Revision History	2
Product Summary	9
Product Name	9
Product Slogan	9
Functional Requirements	9
Priority 1:	9
Priority 2:	11
Unique Features	13
Feature 1: Academics Map Integration	13
Feature 2: Visual Degree Progress Tracking	13
Feature 3: Interactive Degree Planner	13
Deployment URL	13
Usability Test Plan	14
Usability Test Form	14
Test Objectives	14
Task 1:	14
Task 2:	14
Task 3:	15
Task 4:	15
Task 5:	16
Test Setup and User Profile	17
Effectiveness & Efficiency	17
User Satisfaction Survey	18
User Satisfaction Survey Questions	18
User Satisfaction Survey Analysis	18
QA Test Plan	19
Performance	19
Reliability	20
Security	21
Compatibility	22
Robustness	23
AI Feature Testing and Ethics Review	24
Localization Testing	25
Objective:	25
Test Setup:	25
Test Case Results:	25
Code Review	30
Security Self Check	34
Protected Assets	34
Password Hashing Strategy	34

Input Validation.....	35
Account Activation Validation.....	35
Signup Form Validation.....	35
Login Validation.....	36
Protected API Requests.....	36
Account Invitation Validation.....	37
Search Input Validation.....	38
Search Text Cleaning.....	38
Search Filter Validation.....	38
Parameterized Search Query.....	38
Non Functional Requirement Status.....	39
1. Performance.....	39
2. Reliability.....	40
3. Usability.....	41
4. Security.....	42
5. Privacy.....	43
6. Compatibility.....	43
7. Hardware and Networking Requirements.....	44
8. Media Content Requirements.....	44
9. Legal, Licensing, and External Requirement.....	45
10. Organizational and Development Constraints.....	45
11. Coding Standards.....	46
Team Contributions.....	48
Contribution Details.....	48

Product Summary

Product Name

BigBacks Edu

Product Slogan

The most efficient management solution for modern academia. Built with administrators and students in mind.

Functional Requirements

Priority 1:

Authentication / User

- FR 1.1: A User shall be able to log in with an email and password.
- FR 1.2: A User shall be able to log out of their account.
- FR 1.3: A User shall be identified as either a Student or an Admin.
- FR 1.4: A User shall be shown features based on their assigned role.

Student Profile

- FR 2.1: A Student shall have an academic level, such as freshman, sophomore, junior, senior, or graduate.
- FR 2.2: A Student shall have a student type, such as first-time student, continuing student, transfer student, or international student.
- FR 2.3: A Student shall be able to view their profile information.
- FR 2.4: A Student shall be able to update editable profile information.
- FR 2.5: A Student shall have a walking speed value used for walking time calculations.

Course Search

- FR 3.1: A Student shall be able to search for Courses.
- FR 3.2: A Student shall be able to filter Courses by available search filters.
- FR 3.3: A Student shall be able to view Course details.
- FR 3.4: A Course shall have basic course information, including course code, title, units, and description.
- FR 3.5: A Course shall be associated with a Department.
- FR 3.6: A Course shall show available Class Sections for a selected Semester.

Class Section

- FR 4.1: A Class Section shall have section details, including section number, term, instructor name, meeting time, location, enrollment status, and seat availability.
- FR 4.2: A Class Section shall have a modality, such as in-person, online, or hybrid.
- FR 4.3: A Class Section shall have a meeting type, such as synchronous or asynchronous.
- FR 4.4: A Class Section shall be addable to a Student's Cart when eligible.
- FR 4.5: A Class Section shall indicate whether it conflicts with Class Sections already in a Student's Cart or Schedule.

Degree Planner

- FR 5.1: A Student shall be able to view a Degree Planner based on their declared Major.
- FR 5.2: A Student shall be able to add Courses to future academic terms in their Degree Planner.
- FR 5.3: A Student shall be able to remove Courses from future academic terms in their Degree Planner.
- FR 5.4: A Degree Planner shall store planned Courses by academic year and term.
- FR 5.5: A Degree Planner shall display course code, course title, units, and requirement category for each planned Course.
- FR 5.6: A Degree Planner shall allow a Student to move Courses between terms.
- FR 5.7: A Degree Planner shall suggest Courses based on remaining requirements and completed prerequisites.

Semester

- FR 6.1: A Semester shall show term name, start date, end date, add/drop deadline, withdrawal deadline, and enrollment period.
- FR 6.2: A Semester shall indicate whether it is the active term.

Department

- FR 7.1: A Department shall have a department name, office location, and contact information.
- FR 7.2: A Department shall show a list of associated Courses.

Notifications

- FR 8.1: A Student shall be able to view Notifications.
- FR 8.2: A Student shall be able to dismiss Notifications.
- FR 8.3: A Notification shall inform a Student when their enrollment period begins.
- FR 8.4: A Notification shall inform a Student about approaching academic deadlines.
- FR 8.5: A Notification shall warn a Student about schedule conflicts or insufficient walking time between classes.
- FR 8.6: The system shall display feedback when a Student action succeeds or fails.

Priority 2:

Schedule

- FR 9.1: A Schedule shall display a Student's Cart and enrolled Class Sections in a weekly format.
- FR 9.2: A Schedule shall display each Class Section on the correct day and time.
- FR 9.3: A Schedule shall visually distinguish Cart classes from enrolled classes.
- FR 9.4: A Schedule shall display relevant course information, including course code, meeting time, instructor name, building, and room.
- FR 9.5: A Schedule shall allow a Student to select a Class Section and view its Course details.
- FR 9.6: A Schedule shall allow a Student to select a class location and open it on the Campus Map.
- FR 9.7: A Schedule shall detect time conflicts between selected Class Sections.
- FR 9.8: A Schedule shall warn a Student when estimated walking time between back-to-back Class Sections exceeds the available time gap.

Cart

- FR 10.1: A Student shall be able to view their Cart.
- FR 10.2: A Student shall be able to add Class Sections to their Cart.
- FR 10.3: A Student shall be able to remove Class Sections from their Cart.
- FR 10.4: A Cart shall store Class Sections selected by a Student.
- FR 10.5: A Cart shall display relevant course information, including course name, course number, units, instructor name, meeting time, location, and enrollment status for each section.
- FR 10.6: A Cart shall calculate the total number of selected course units.
- FR 10.7: A Cart shall show schedule conflict and walking time warnings for selected Class Sections.

Degree Progress

- FR 11.1: A Student shall be able to view Degree Progress based on their declared Major.
- FR 11.2: A Student shall be able to update their declared Major for major planning purposes.
- FR 11.3: A Degree Progress shall display required General Education courses, major electives, and major requirements.
- FR 11.4: A Degree Progress shall display completed, planned, and remaining degree requirements.
- FR 11.5: A Degree Progress shall display remaining units needed for graduation.
- FR 11.6: A Degree Progress shall display prerequisite information for planned Courses.
- FR 11.7: A Degree Progress shall recommend Courses that satisfy remaining requirements.

- FR 11.8: A Degree Progress shall allow a Student to view which Courses can satisfy remaining requirements.
- FR 11.9: A Degree Progress shall update when a Student updates their Degree Planner or completed coursework.
- FR 11.10: A Degree Progress shall update when a Student changes their declared Major.
- FR 11.11: A Degree Progress shall display which requirement category a planned Course satisfies.

Admin Management

- FR 12.1: An Admin shall be able to add, remove, or update Course information.
- FR 12.2: An Admin shall be able to add, remove, or update Class Section information.
- FR 12.3: An Admin shall be able to add, remove, or update Building location information.
- FR 12.4: An Admin shall be able to add, remove, or update Degree Requirement information.
- FR 12.5: An Admin shall be able to modify Student profile information.
- FR 12.6: An Admin shall be able to manage Semester information.
- FR 12.7: An Admin shall be able to send Notifications to one Student or all Students.

Major

- FR 13.1: A Major shall have a major name, related Department, required units, and catalog year.
- FR 13.2: A Major shall show its Degree Requirements.

Degree Requirement

- FR 14.1: A Degree Requirement shall have a requirement category, required units, and requirement status.
- FR 14.2: A Degree Requirement shall be satisfied by Courses.
- FR 14.3: A Degree Requirement shall indicate whether the requirement is completed, planned, or remaining.

Campus Map

- FR 15.1: A Student shall be able to view class building locations on the Campus Map.
- FR 15.2: A Campus Map shall display campus building locations.
- FR 15.3: A Campus Map shall display the building location for a selected Class Section.
- FR 15.4: A Campus Map shall display classroom location information for a selected Class Section.
- FR 15.5: A Campus Map shall display campus resources, such as the library, gym, student center, and academic buildings.
- FR 15.6: A Campus Map shall allow a Student to return to the Schedule after viewing a class location.

Unique Features

Feature 1: Academics Map Integration

The **Academics** page allows students to search for courses while viewing their locations on an integrated campus map. Selecting a class highlights where its building is located on the map and provides relevant building details.

Why is it unique? Most student portals do not have interactive campus maps or rely on third-party map applications. Our product allows students to see course information and class locations without switching between pages or applications. This makes course selection easier for students unfamiliar with their campus.

Feature 2: Visual Degree Progress Tracking

The **Degree Progress** page gives students a clear breakdown of their graduation requirements. It shows completed, in-progress, planned and remaining requirements across categories such as major courses, major electives, general education, and total units. Our page features a long progress bar at the very top of the page to indicate what percentage completion the student is on.

Why is it unique? Most student center applications do not organize degree progress information in a way that's clear and straight-forward. Our application reorganizes this information into distinct categories with clear indicators for courses of varying completion status. Most student center applications also do not show how close a student is to completion of their degree in a percentage view.

Feature 3: Interactive Degree Planner

The **Degree Planner** page organizes a student's remaining requirements beside a multi-year roadmap. Students can add future semesters (inc. Fall, Winter, Spring, and Summer), drag and drop courses into them to build their entire academic journey. Students can adjust their plan anytime as needed.

Why is it unique? Our competitors often incorporate a tedious style of degree planning such as jumping through multiple pages and selections to add a course to the semester. Our application is extremely easy to use with our drag-and-drop design. The entirety of a student's major roadmap is displayed on the left side, all labeled with the courses' categories. This means the student has every tool to shape their unique academic journey. Academic advising is still recommended, of course.

Deployment URL

<http://18.226.97.234/>

Usability Test Plan

Link to our usability test form: <https://forms.gle/F2RfecyiVhYUxxGSA>

Our 5 testing objectives:

Task 1:

TASK	DESCRIPTION
Task	Set Walking Speed and Preferred Name
Application State	The user is on the homepage while logged into a student account.
Successful Completion Criteria	The user sets a preferred name and sets walking speed in meters per second. The saved information should remain visible after updating.
Benchmark	Complete in 3 minutes

Task 2:

TASK	DESCRIPTION
Task	Search for courses and add class section(s) to Cart
Application State	The user is on the homepage while logged into a student account.
Successful Completion Criteria	From the public home page, navigate to Course Search, search for a specific department such as CSC, choose a course section, and add it to the cart or schedule.
Benchmark	Complete in 5 minutes

Task 3:

TASK	DESCRIPTION
Task	Search for courses and add class section(s) to Cart
Application State	Logged-in student account on the home page.
Successful Completion Criteria	From the public home page, navigate to Course Search, search for a specific department such as CSC, choose a course section, and add it to the cart or schedule.
Benchmark	Complete in 5 minutes

Task 4:

TASK	DESCRIPTION
Task	Trigger a Scheduling Conflict or Walking-Time Alert
Application State	Logged-in student account with walking speed already set.
Successful Completion Criteria	The user adds class sections that cause the system to display either a scheduling conflict or walking-time alert. The user should be able to notice the alert after it appears.
Benchmark	Complete in 10 minutes

Task 5:

TASK	DESCRIPTION
Task	Check out Notifications
Application State	Logged-in student account on the home page with at least one notification available.
Successful Completion Criteria	The user reads a notification, understands what it is informing them about, and dismisses it successfully.
Benchmark	Complete in 2 minutes

Test Setup and User Profile

Before testing, we made sure our application is as polished as possible. We specifically focused on making sure our unique features work, and that our application is as secure as we can make it by identifying any source of vulnerability and eliminating them. As a result, we did not start testing until about the halfway point of the milestone.

As our application requires users' emails to be authenticated before they can sign up, verifying each tester's email is too time-consuming. The solution we came up with was to make an auto-authentication system for any email domain ending in @bbedu.com. Other solutions we brainstormed were 1. to make a shared account but we decided it would be too chaotic and 2. to make 80+ accounts beforehand but we determined it would be too difficult to track and ration.

Effectiveness & Efficiency

(as of 8:55 PM 7/27 Mon)

Task	% Completed	Errors	Comments	Efficiency (in minutes) min, max
1	85.7%	Preferred name wouldn't save	N/A	0.5 - 3
2	89.3%	N/A	Dragging courses from year to year is not convenient	0.5 - 6
3	96.4%	N/A	Academics page could do with a better name	1 - 10
4	85.7%	Adding two classes in the same building won't trigger walking-time conflict	Schedule-conflict warning should show before adding courses to cart in addition to after	0.5 - 5
5	92.9%	N/A	N/A	0.05 - 6

User Satisfaction Survey

User Satisfaction Survey Questions

1. **Task 1: Set Walking Speed and Preferred Name**
 - 1.1. Setting my walking speed and preferred name was easy.
 - 1.2. Setting my walking speed and preferred name was easy.
 - 1.3. The time it took to set my walking speed and preferred name was reasonable.
2. **Task 2: Plan First-Year Courses, Including Optional Winter and Summer Terms**
 - 2.1. Planning my first-year courses was easy.
 - 2.2. The degree planning feature for adding courses and optional semesters was easy to find.
 - 2.3. The time it took to plan my first-year courses was reasonable.
3. **Task 3: Search for Courses and Add Class Section(s) to Cart**
 - 3.1. Searching for courses and adding a class section to the Cart was easy.
 - 3.2. The course search and Cart features were easy to find.
 - 3.3. The time it took to search for a course and add a section to the Cart was reasonable.
4. **Task 4: Trigger a Scheduling Conflict or Walking-Time Alert**
 - 4.1. Triggering a scheduling conflict or walking-time alert was easy.
 - 4.2. I was able to find and notice the alert after it appeared.
 - 4.3. The time it took to trigger and review the alert was reasonable.
5. **Task 5: Read and Dismiss Notification(s)**
 - 5.1. Reading and dismissing a notification was easy.
 - 5.2. The notification feature was easy to find.
 - 5.3. The time it took to read and dismiss a notification was reasonable.

User Satisfaction Survey Analysis

Link to Survey Results:

<https://docs.google.com/forms/d/e/1FAIpQLSeeG0Q4lN6O-Zk9LTBUdQT9DxPWJAK5IDEFq74K7e51sSbxaw/viewanalytics>

Most people were able to accomplish our survey tasks. This is encouraging from a usability standpoint as we did not give them any direction other than actions they need to achieve. There was one obvious troll responder who may have brought the % completed rate down but the average completion rate across all 5 tasks remains at 90%.

Task 3 had the most completion, but it also had the most detailed complaints. People struggled to find the features and the UI is not polished enough. Task 5 had the strongest satisfaction results where no users selected strongly disagree or disagree; task 4 had the weakest satisfaction results where 8 users selected neutral. Task 4 also had the least completion together with Task 1.

Another notable stat was the median completion time being 8 minutes. Since our usability test plan was intended to take around 10 minutes, this supports that our application is decently navigable and that most users were able to complete the tasks within the expected time frame.

QA Test Plan

Performance

P-#	Test Case	Test Stept	Expected	Result	P/NP
P-01	The system shall display a visual response within three seconds after a user submits a normal request under expected class-project usage.	Log in as Student, search for CSC, and measure the response time using Chrome DevTools	Response In 3 second	(3 Run) 162/99/71(ms)	Pass
P-02	The class schedule page shall load within three seconds under normal network conditions.	Log in as Student, tap schedule button, and measure the response time using Chrome DevTools	Response In 3 second	(3 Run) Calendar: 185/111/114(ms) Walking-warnings: 194/111/112(ms) Semesters: 194/111/108(ms) Map: 394/395/382(ms)	Pass
P-03	The Admin should be available to change the active Semester.	Log in as Admin, on the Dashboard active another semester.	The semester should be saved. Students should not edit the not active semester.	When I changed the Semester to 2026 fall, it was successful. But after I open another browser to login as a student, and then come back, the semester will auto return back to 2026 summer.	Not Pass

Reason for P-03 not passing: Every time I leave the Dashboard and return, the component remounts, so semesterChoice reverts to an empty string. When the old code detects that it is empty, it automatically falls back to Active semester; If there is no active semester, it uses the first item in the semester list.

How did we fix it?

const selectedSemesterId = semesterChoice; We limit the drop-down menu to only the "semesterChoice" explicitly selected by the administrator; it will not automatically select the "active semester" or the first option simply because the user has accessed the Dashboard.

Reliability

R-#	Test Case	Test Stept	Expected	Result	P/NP
R-01	The system shall display a clear error message when an operation cannot be completed.	Enter a valid Student email with an incorrect password, then click Log In	The website will display the message : Invalid email or password	Invalid email or password	Pass
R-02	A failed database operation shall not create incomplete or duplicate records.	Admin login and crate a exist Courses	Can not create that Course	A record with the same unique values already exists.	pass
R-03	The system shall confirm successful create, update, and delete operations visually to the user.	Admin login and create a class, then change the class info and then delete it.	Course create successful and update successful, and delete successful	When we're doing the delete, the program won't let me do it.	Not Pass

Reason for R-03 not passing: In the previous implementation, the course deletion logic treated a course as if it were an independent record. However, `course_id` is referenced as a foreign key by several other tables, including degree requirements, prerequisites, degree roadmaps, student plans, and class sections.

When we attempted to delete a course directly, PostgreSQL rejected the operation because the related records would still reference a `course_id` that no longer existed. This would create orphaned records and break referential integrity.

How did we fix it?

We updated the deletion process to handle the related records in the correct dependency order.

First, the system selects and locks the requested course. It then checks whether the course has any associated class sections. If class sections exist, the deletion is stopped and the administrator is asked to remove those sections first, because they may contain important scheduling or student-related data.

If no class sections exist, the system removes the course's related records from student plans, degree roadmaps, planned courses, degree requirements, prerequisite relationships, and GE-area mappings. After all safe dependencies have been removed, the course itself is deleted.

The entire operation is performed inside a database transaction. If any step fails, all previous changes are rolled back, preventing the database from being left in a partially deleted or inconsistent state.

Security

S-#	Test Case	Test Stept	Expected	Result	P/NP
S-01	User has to wait 6 second to sent a same request	Login as Student, in Search Course page press search	After the press the button become non pressable and renew after 6 second	After the press the button become non pressable give the message please wait, and then it renew after 6 second	Pass
S-02	When user logout they can't access the logged page by typing the ip link in browser	Copy the Link, then logout, and pase the link see if we can go back to logged page.	It going to return to login page	It returned to login page	Pass
S-03	Once a request in to cooldown, the other request still allowed	Press the Search button and then press the other button see if it still works	The Search request remains in cooldown, but the Add to Cart request is still accepted.	The Search button remained in cooldown, while Add to Cart successfully added the class to the cart.	pass

Compatibility

C-#	Test Case	Test Stept	Expected	Result	P/NP
C-01	Test if the program can work will with Chrome Browser	Login as student, search course, add section to cart, check Schedule.	Display and functional all woks well	Everything looks well and works well	Pass
C-02	Test if the program can work will with Microsoft Edge	Login as student, search course, add section to cart, check Schedule.	Display and functional all woks well	Everything looks well and works well	Pass
C-03	Test if the windowed Browser can fine views	Login and check each pages. Find if there is any block or display error	Display should works well, smaller the windows will not make any button or logo or nav bar despair	Smaller the window, there will have the slide bar on the right and bottom, but the mean button and nav bar still display fine	pass

Robustness

E-#	Test Case	Test Stept	Expected	Result	P/NP
E-01	Incorrect Username or Password display	Go to login page type in incorrect Username and Password	Display the error message under the password box Invalid email or password.	Display the error message "Invalid email or password."	Pass
E-02	Empty password or user name login	Go to login page and Leave the username and password box empty and try to login	Display the error message under each Username box and Password box	Message display: E-mail is required. Password is required.	Pass
E-03	Search a not exist Class	Log in and Search a class not in database	The course search page should display a chat box says "No courses found"	The course search displayed a chat box says "No courses found"	pass

AI Feature Testing and Ethics Review

The prototype of our student center application does not utilize any AI-assisted or AI-powered feature. Many of our academic-related suggestions are generated through rule-based logic using information from several tables of our database. In no capacity does the system use an LLM or machine learning model to make decisions.

Since no AI model is integrated, AI-related risks such as hallucination or biased suggestions do not apply to our application. As for ethical risks, course suggestions of our application are presented as planning aids only and should not replace official academic advising.

If we were to expand the project in the future, AI-powered features would be used only as an assistant to help explain options, not as a decision maker for enrollment, graduation eligibility, account approval, or degree completion.

Localization Testing

Objective:

BigbacksEdu is currently built as an English only, US-locale application. Our primary goal for localization testing is to verify that assumption by hands-on testing that assumption.

We tested for:

1. UI text can be retrieved from a translatable source (Vs hardcoded string).
2. Whether dates, times, and numbers adapt to non US locale settings.
3. Whether our website layout can support longer translated strings and right to left text direction.
4. Whether the page declares a language to the browser.

Test Setup:

Browser: Google Chrome, tested against our prototype (18.226.97.234).

Method: Chrome's displayed language was changed using (chrome://settings/languages) to Arabic (ar-SA, RTL) and Spanish (es), to confirm whether our UI follows the browser's active language. Long text layout tolerance was also tested by using the inspect button with Chrome's DevTools to simulate a longer translated string.

Application State: The pages tested were our FAQ page, and the student logged-in portal home page because we got to test both a public/authenticated view.

Test Case Results:

T-#	Test Case	Test Steps	Expected Result	Result	P/NP
T-01	Static Page Language Declaration	Inspect the pages <html tag>	Lang attribute should reflect the active locale	<html lang="en"> is hardcoded in index.html	NP
T-02	UI text translation	Set Chrome's display language to Arabic, and also Spanish; reload the FAQ page and the student portal.	Labels and content should appear in the active language.	With Chrome set to both Arabic and Spanish. The website still remained fully in English across all of our webpages.	NP

T-03	Layout with Longer Translated Texts	In Devtools, edited the "View Cart" button text to a longer French Text to simulate translation length.	Buttons should wrap or resize based on the length of the text.	The button sits inside a flex-based container, so the text is wrapped cleanly automatically.	P
T-04	Right to Left layout Support	Set Chrome's display language to Arabic and reloaded the FAQ page.	Layout should mirror horizontally when a RTL language is active.	Chrome's own browser correctly mirrored RTL but our app was unable to mirror and text alignment was still LTR.	NP

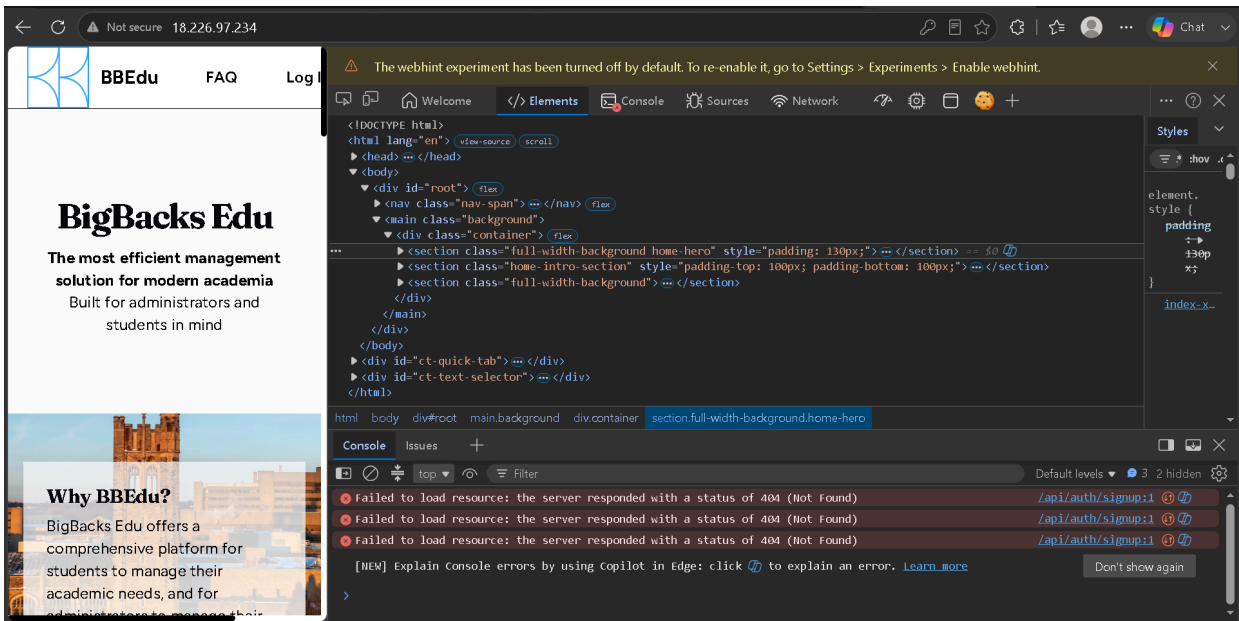
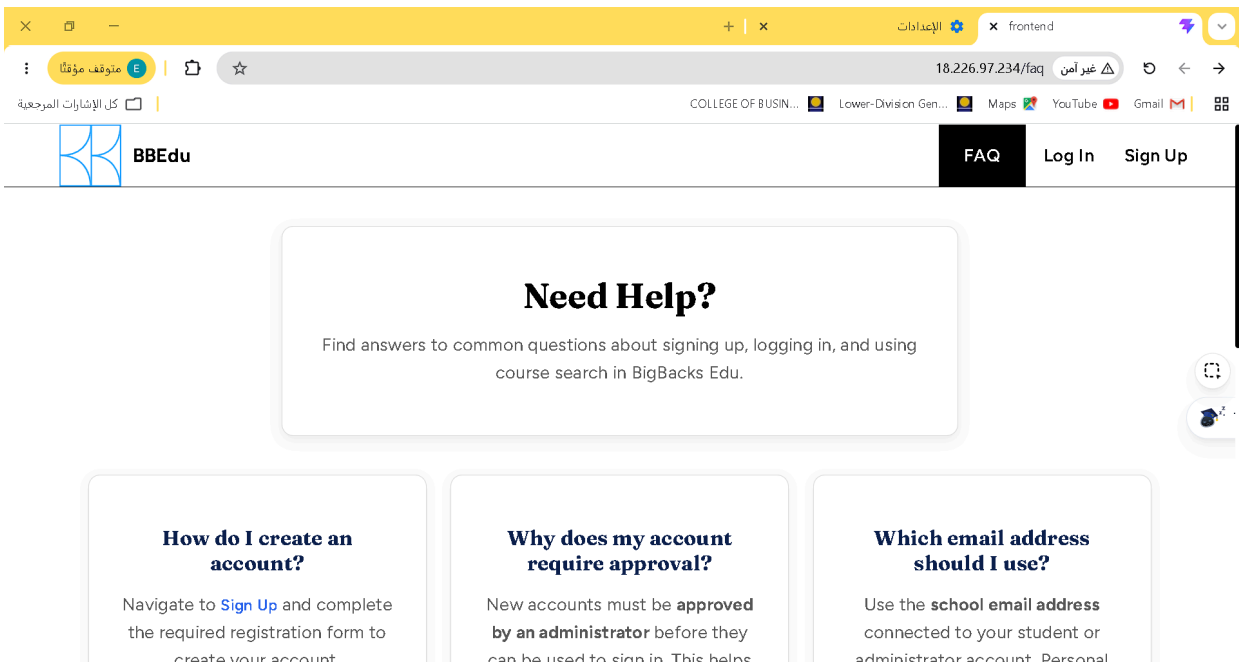
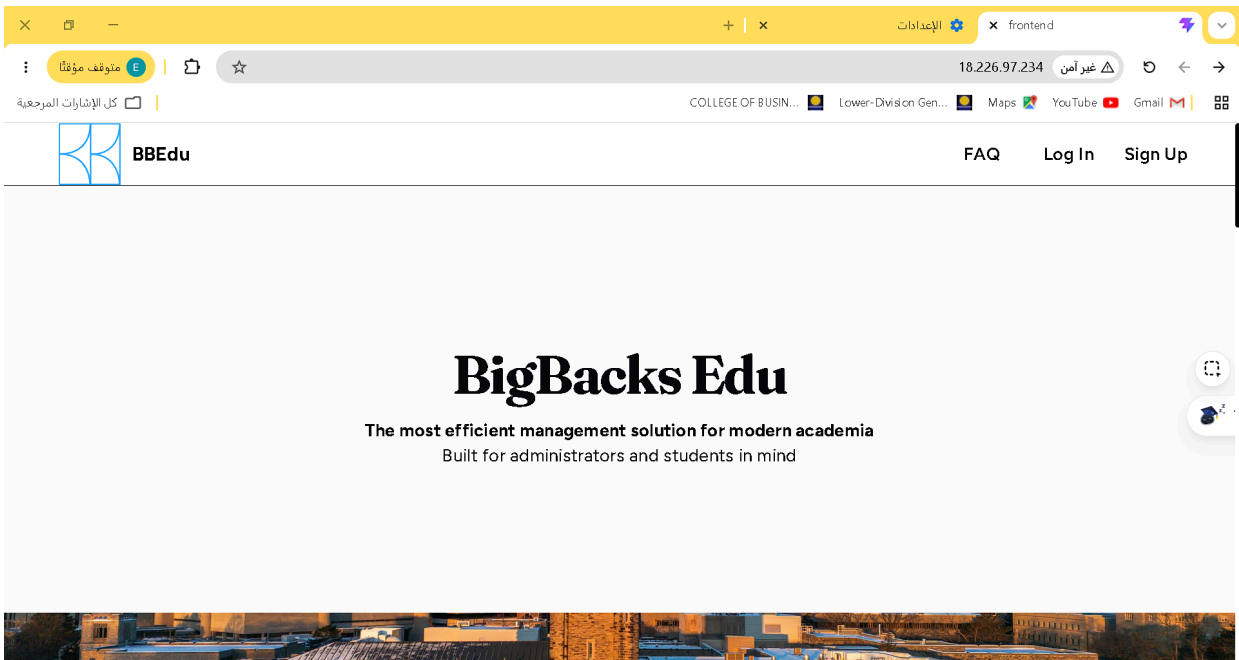
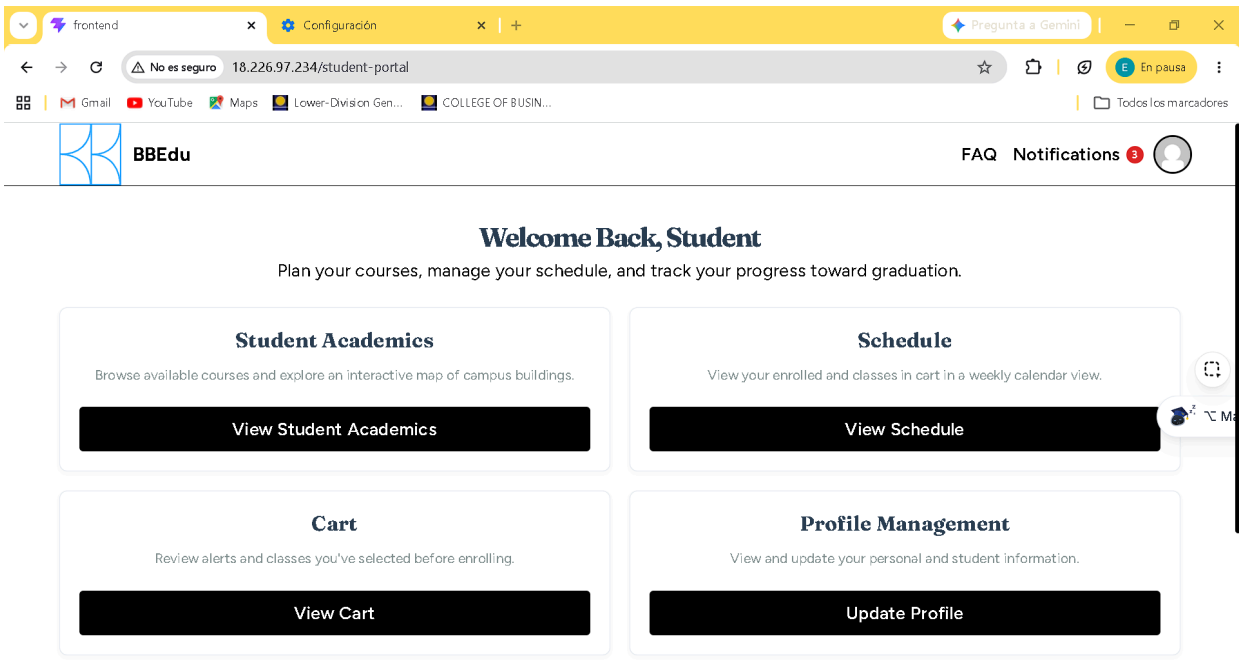


Image: T-01 (Static Page Language Declaration)





Images: T-02 (UI Text Translation)

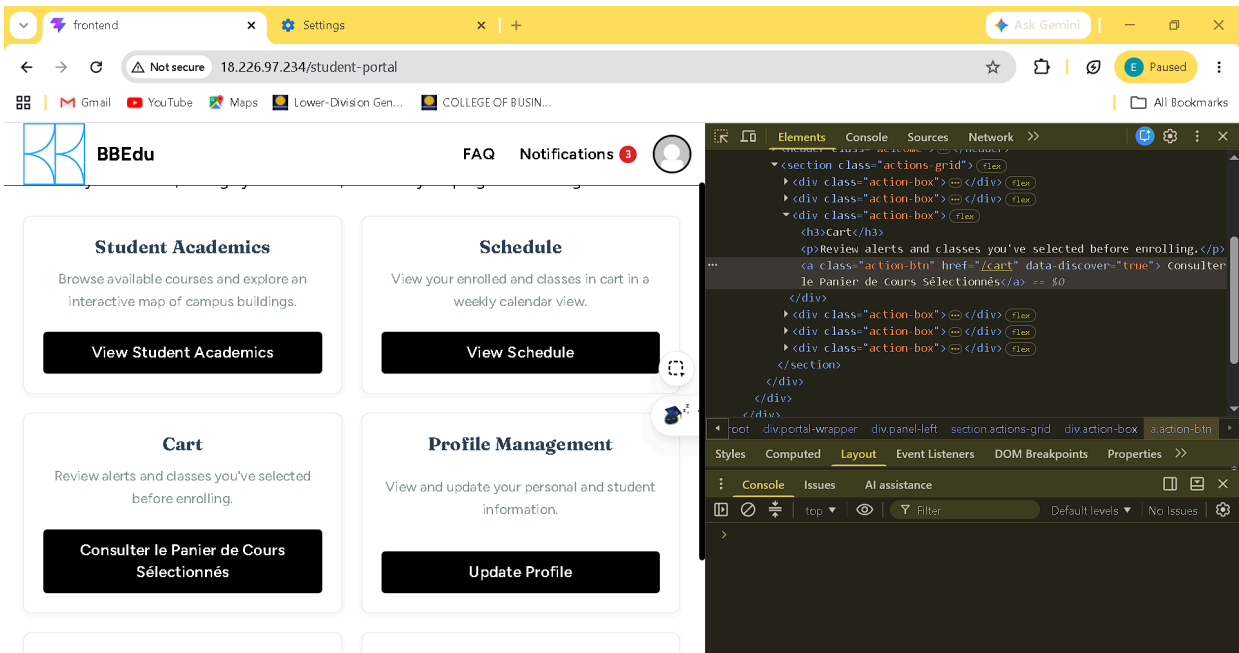


Image: T-03 (Layout with Longer Translated Texts)

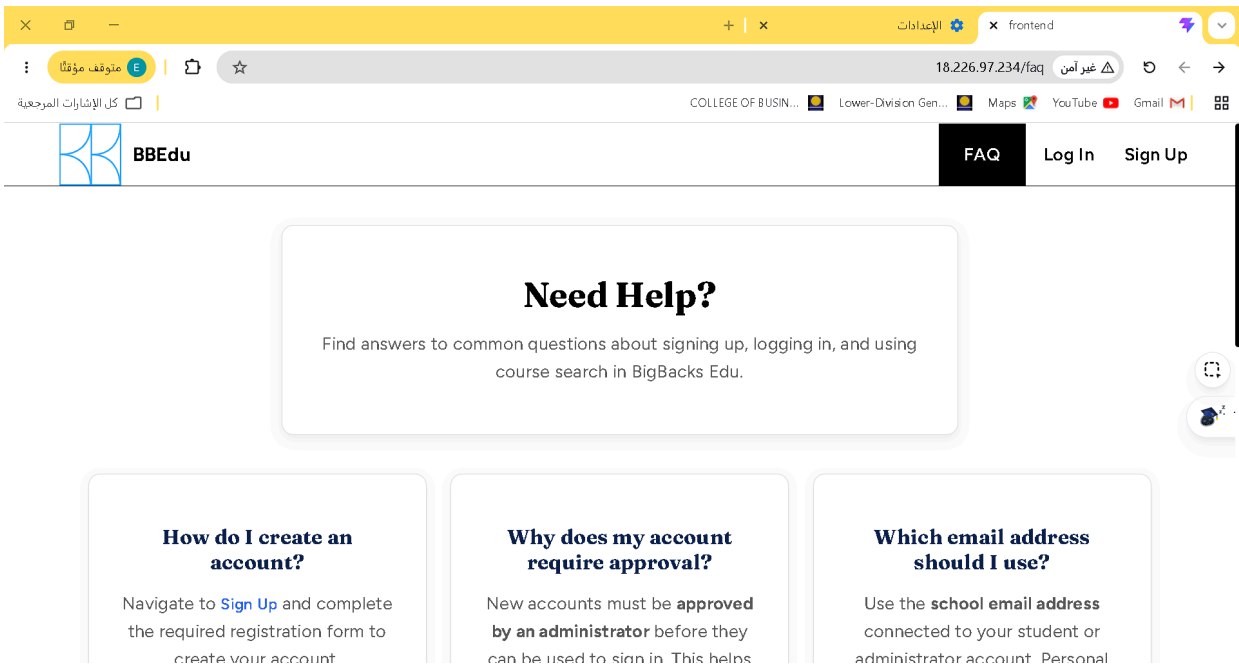
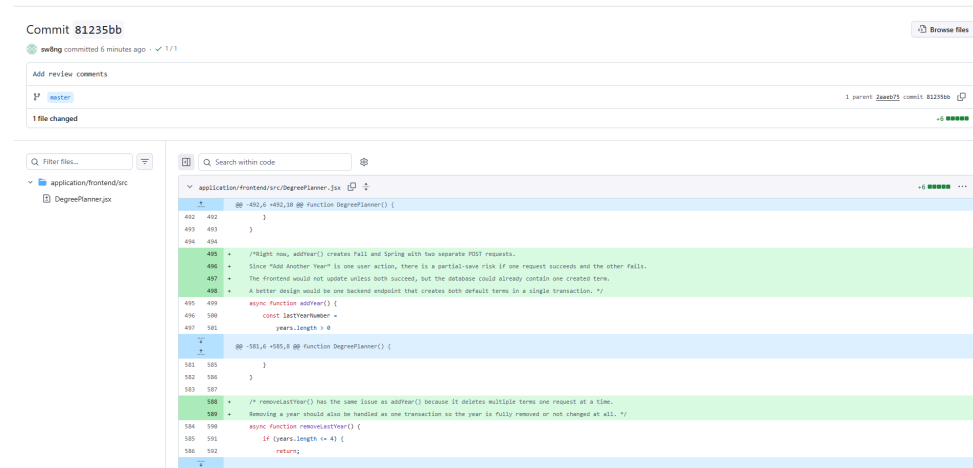


Image: T-04 (Right to Left Layout Support)

Code Review

Internal Code



Comments by Sean:

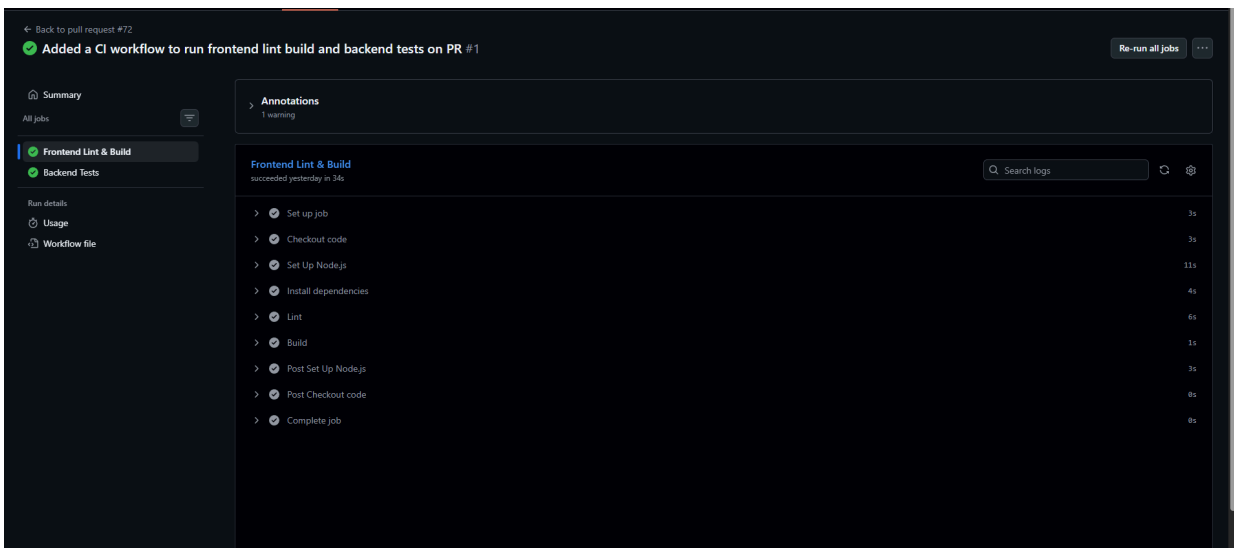
Right now, `addYear()` creates Fall and Spring with two separate POST requests. Since “Add Another Year” is one user action, there is a partial-save risk if one request succeeds and the other fails.

The frontend would not update unless both succeed, but the database could already contain one created term.

A better design would be one backend endpoint that creates both default terms in a single transaction.

`removeLastYear()` has the same issue as `addYear()` because it deletes multiple terms one request at a time.

Removing a year should also be handled as one transaction so the year is fully removed or not changed at all.



CI Workflow Integration by Enoch:

I added the CI workflow to support code review and added a github actions workflow that automatically runs after every pull request. The screenshot above shows the workflow passing for both frontend lint and backend on PR#72, confirming that the checks are running correctly and giving code reviewers an easier process to determine pass/fail before approving a merge.

External Code Review for Team 1 (TerrorByte-01)

FinanceService has `totalIncome()` and `totalExpenses()` that filter out null amounts before summing (`.filter(i -> i.getAmount() != null)`), but that same income/expense calculation is duplicated in three other places: `RecommendationService`, `TrueCostCalculatorService`, and `SubscriptionService`, without the null filter. Since amount is a nullable Double on both income and expense, any record with a null amount will throw a `NullPointerException` in those three services, even though `FinanceService` handles the same case safely. `FinanceService`'s own class comment already flags this duplication as tech debt and recommends the other services call it directly instead of recalculating inline.

File: FinanceService.java (lines 116–122) — correct version, filters null amounts


```

115
116 // Requirement 1.4: summary of income and expenses.
117 public double totalIncome(Long userId) {
118     return incomeRepository.findByUserId(userId).stream()
119         .filter(i -> i.getAmount() != null)
120         .mapToDouble(Income::getAmount)
121         .sum();
122 }

```

File: RecommendationService.java (lines 65–78) — same calculation duplicated, missing the null filter

```

64 // falls back to a deterministic rule-based generator so the feature still works.
65 public List<Recommendation> generateSubscriptionRecommendations(User user) {
66     Long userId = user.getUserId();
67
68     double totalIncome = incomeRepository.findByUserId(userId).stream()
69         .mapToDouble(Income::getAmount).sum();
70     double totalExpenses = expenseRepository.findByUserId(userId).stream()
71         .mapToDouble(Expense::getAmount).sum();
72     List<Subscription> active = subscriptionRepository
73         .findByUserIdAndStatus(userId, "active");
74     double subscriptionCost = active.stream()
75         .mapToDouble(Subscription::getMonthlyCost).sum();
76
77     double availableMoney = totalIncome - totalExpenses - subscriptionCost;
78 }

```

File: TrueCostCalculatorService.java (lines 23–27) — same issue duplicated again

```

23 public TrueCostResult calculateImpact(User user, Goal goal, Double purchaseAmount) {
24     double totalIncome = incomeRepository.findByUserId(user.getUserId()).stream()
25         .mapToDouble(Income::getAmount).sum();
26     double totalExpenses = expenseRepository.findByUserId(user.getUserId()).stream()
27         .mapToDouble(Expense::getAmount).sum();

```

Security Self Check

Protected Assets

Our application's protected assets include: login credentials, hashed passwords, authentication tokens, student profile information, student schedules, cart data, degree planner data, course planning records, and admin management functions.

Admin-only actions such as creating courses, class sections, buildings, semesters, and notifications are protected so they cannot be accessed by normal student users.

Password Hashing Strategy

Passwords are not stored in plain text. During account activation, the backend takes the submitted password and passes it through `hashPassword(password)` before inserting the user into the `app_user` table.

```
const created = await client.query(  
  [  
    "INSERT INTO public.app_user",  
    "(school_id, username, institutional_email, password_hash,",  
    "first_name, last_name, user_role, created_at)",  
    "VALUES ($1, $2, $3, $4, $5, $6, $7, CURRENT_TIMESTAMP)",  
    "RETURNING user_id, username, institutional_email,",  
    "first_name, last_name, user_role",  
  ].join(" "),  
  [  
    school.rows[0].school_id,  
    username,  
    invitation.institutional_email,  
    hashPassword(password),  
    invitation.first_name,  
    invitation.last_name,  
    invitation.user_role,  
  ]  
)
```

File: accountAuth.js

```
function hashPassword(password) {  
  const salt = crypto.randomBytes(16).toString("hex");  
  const hash = crypto  
    .scryptSync(password, salt, PASSWORD_KEY_LENGTH)  
    .toString("hex");  
  
  return `scrypt:${salt}:${hash}`;  
}
```

File: auth.js

Input Validation

Account Activation Validation

The frontend validates required fields before sending the signup request. It checks that the institutional ID is present, the email has a valid format, the confirmation email matches, and the password is at least 8 characters.

```
if (
  !externalId ||
  !institutionalEmail ||
  typeof password !== "string" ||
  password.length < 8
) {
  return res.status(400).json({
    message:
      "id, institutionalEmail, and an 8+ character password are required.",
  });
}
```

File: accountAuth.js

Signup Form Validation

Just like account activation, the frontend validates required fields before sending the signup request. It checks that the institutional ID is present, the email has a valid format, the confirmation email matches, and the password is at least 8 characters.

```
function validate() {
  const newErrors = {};

  if (!id.trim()) {
    newErrors.id = "Student ID is required.";
  }

  if (!email.trim()) {
    newErrors.email = "Email is required.";
  } else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    newErrors.email = "Enter a valid email address.";
  }

  if (!confirmEmail.trim()) {
    newErrors.confirmEmail = "Please confirm your email address.";
  } else if (
    confirmEmail.trim().toLowerCase() !== email.trim().toLowerCase()
  ) {
    newErrors.confirmEmail = "Email addresses do not match.";
  }

  if (!password) {
    newErrors.password = "Password is required.";
  } else if (password.length < 8) {
    newErrors.password = "Password must be at least 8 characters.";
  }

  return newErrors;
}
```

File: Signup.jsx

Login Validation

The login form validates that the user enters an email and password before submitting to the backend

```
function Login() {
  const navigate = useNavigate();

  const [identifier, setIdentifier] = useState("");
  const [password, setPassword] = useState("");
  const [errors, setErrors] = useState({});
  const [submitting, setSubmitting] = useState(false);
  const [serverMessage, setServerMessage] = useState("");

  function validate() {
    const newErrors = {};

    if (!identifier.trim()) {
      newErrors.identifier = "E-mail is required.";
    }

    if (!password) {
      newErrors.password = "Password is required.";
    }

    return newErrors;
  }
}
```

File: Login.jsx

Protected API Requests

Our application uses `authFetch()` to attach the user's authentication token to protected backend requests. If the backend returns 401 or 403, the frontend handles expired sessions or unauthorized access.

```
export async function authFetch(input, init = {}) {
  const token = localStorage.getItem("authToken") || "";
  const response = await fetch(input, {
    ...init,
    headers: {
      ...(init.body && !(init.body instanceof FormData) ? { "Content-Type": "application/json" } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...(init.headers || {}),
    },
  });
}
```

File: AuthContext.jsx

Account Invitation Validation

Account creation only goes through if the institutional ID and institutional email match an available account invitation. This prevents unauthorized users from creating unapproved accounts.

```
try {
  await client.query("BEGIN");

  const result = await client.query(
    `SELECT *
     FROM public.account_invitation
     WHERE external_id = $1
        AND LOWER(institutional_email) = LOWER($2)
     FOR UPDATE`,
    [externalId, institutionalEmail]
  );

  const invitation = result.rows[0];

  if (!invitation || invitation.claimed_at) {
    await client.query("ROLLBACK");

    return res.status(404).json({
      message: "An available account invitation was not found.",
    });
  }
}
```

File: accountAuth.js

Search Input Validation

Search Text Cleaning

The backend cleans text-based search inputs before using them. Search values such as course search, subject code, course number, modality, status, meeting type, level, section type, and course category are trimmed before processing.

```
function cleanText(value) {  
  return String(value || "").trim();  
}
```

File: courses.js

Search Filter Validation

The backend validates filters before running the course search query. Numeric filters such as departmentId, semesterId, geAreaId, and units must be valid numbers. Course category must also match an approved value such as major-core, major-elective, or ge. Invalid filters return a 400 error response.

```
function cleanId(value) {  
  const id = Number(value);  
  return Number.isInteger(id) && id > 0 ? id : null;  
}
```

File: courses.js

```
if (  
  (req.query.departmentId && !departmentId) ||  
  (req.query.semesterId && !semesterId) ||  
  (req.query.units && (!Number.isInteger(units) || units <= 0)) ||  
  (req.query.geAreaId && !geAreaId) ||  
  (courseCategory && ![ "major-core", "major-elective", "ge" ].includes(courseCategory))  
) {  
  return res.status(400).json({  
    message:  
      "departmentId, semesterId, geAreaId, units, and courseCategory must be valid.",  
  });  
}
```

File: courses.js

Parameterized Search Query

The course search backend does not directly insert user input into the SQL query. Search and filter values are stored in a values array and passed separately into pool.query().

```
function addWhere(where, values, sql, value) {  
  values.push(value);  
  where.push(sql.replace("?", "$" + values.length));  
}
```

File: courses.js

Non Functional Requirement Status

1. Performance

- 1.1. The system shall display a visual response within three seconds after a user submits a normal request under expected class-project usage. **DONE**
 - 1.1.1. **All actions respond quickly (<3 seconds). See QA Tests for examples.**
- 1.2. The login process shall complete within five seconds after valid credentials are submitted. **DONE**
 - 1.2.1. **Login is almost immediate (<1 second) after the user enters valid credentials.**
- 1.3. The class schedule page shall load within three seconds under normal network conditions. **DONE**
 - 1.3.1. **The class schedule page takes 1-2 seconds to load. About 3 seconds exactly with a heavy class load.**
- 1.4. The campus map shall begin displaying within five seconds after a student selects a class location. **DONE**
 - 1.4.1. **The campus map has been changed to be always displayed. Selecting a class section highlights the building on the map instantly (<1 second).**
- 1.5. Degree progress calculations shall be displayed within five seconds after the student opens the Degree Progress page. **DONE**
 - 1.5.1. **Degree Progress is calculated within 3 seconds.**
- 1.6. The system shall support at least 25 simultaneous users during prototype testing without becoming unavailable. **ON TRACK**
 - 1.6.1. **We could not verify that the system supports 25 users at once seamlessly, but through our usability testing we were able to verify the system was able to support around 10 people without issue.**

2. Reliability

- 2.1. The deployed application shall be available through a public URL during schedule demonstration and grading periods. **DONE**
 - 2.1.1. **The deployed EC2 public URL is made available from Milestone 2.**
- 2.2. The system shall not lose previously saved schedule, degree progress, building, course, or office-hour data after a normal server restart. **DONE**
 - 2.2.1. **No data is reset or lost after backend restart.**
- 2.3. The system shall display a clear error message when an operation cannot be completed. **DONE**
 - 2.3.1. **All of our pages display error messages for the user (doubles as debug messages) and some erroneous actions will send notifications or alerts.**
- 2.4. A failed database operation shall not create incomplete or duplicate records. **ON TRACK**
 - 2.4.1. **We identified at least one case of a function that could create incomplete records and promptly fixed the issue.**
- 2.5. The system shall confirm successful create, update, and delete operation visually to the user. **DONE**
 - 2.5.1. **Our application utilizes notifications, pop-up messages and colored text to indicate status of user actions.**
- 2.6. The PostgreSQL database shall be backed up at least once per week during active development. **ISSUE**
 - 2.6.1. **Automatic back-up of the database is not set up. Manual documented backups are made in-between each milestone.**

3. Usability

- 3.1. A first-time student user shall be able to locate their schedule, campus map, degree progress, and planning pages within five minutes without formal training. **DONE**
- 3.2. A student shall be able to open a class location from the schedule in no more than three user actions. **DONE**
 - 3.2.1. **The class location is highlighted when the user selects a class inside of their schedule.**
- 3.3. Navigation labels shall use clear terms such as Schedule, Campus Map, Degree Progress, Course Planning, and Profile. **ON TRACK**
 - 3.3.1. **Several features such as Campus Map and Course Search are combined into one Academics page for usability. Academics isn't a straight-forward page name but we clearly describe the contents of the page inside of the student dashboard.**
- 3.4. Student, Professor, and Admin dashboard shall display only functions relevant to the user's role. **DONE**
 - 3.4.1. **Professor dashboard is scrapped. Student and Admin dashboards only display features pertaining to their respective roles.**
- 3.5. Required form fields shall be clearly marked, and validation errors shall identify the field that needs correction. **DONE**
 - 3.5.1. **Course Search, Log In/Sign Up has suggestions for each field. When the user enters incorrect information there will be an error message below the field.**
- 3.6. The interface shall remain readable at common laptop resolutions, including 1366 x 768 and 1920 x 1080. **DONE**
 - 3.6.1. **Our application is readable and responsive in all reasonable landscape resolutions.**
- 3.7. The system shall provide basic responsive behavior for mobile screens with a width of at least 360 pixels. **ISSUE**
 - 3.7.1. **Our application is not mobile friendly as many of our pages incorporate a side-by-side layout for each page feature. As a result, many of these features would stack on top of each other in mobile view.**
- 3.8. Degree progress shall be separated visually into major requirements, major electives, general education requirements, and total credits. **DONE**
 - 3.8.1. **Our Degree Progress page is exactly as this non-functional requirement describes.**

4. Security

- 4.1. All public communication between the browser and the deployed applications shall use HTTPS. **ISSUE**
 - 4.1.1. **Our deployment lacks an SSL/TLS certificate.**
- 4.2. The application shall use TLS 1.3 where supported by the client and server configuration. **ISSUE**
 - 4.2.1. **Same as 4.1.1.**
- 4.3. User passwords shall be hashed using bcrypt before being stored in the database. **DONE**
 - 4.3.1. **All passwords stored in our database are hashed using bcrypt.**
- 4.4. Plain-text passwords shall never be stored in the database or application log files. **DONE**
 - 4.4.1. **Our database does not store plain-text passwords.**
- 4.5. The system shall require authentication before allowing access to Student, or Admin dashboards. **DONE**
 - 4.5.1. **Our application uses protected routes/token checks for authentication.**
- 4.6. The system shall use role-based authorization to prevent students from accessing Admin functions. **DONE**
 - 4.6.1. **Admin pages are role protected.**
- 4.7. Authentication tokens shall expire after a defined period of inactivity. **ISSUE**
 - 4.7.1. **This is not enforced in our system; there is not inactivity time-out.**
- 4.8. Database credentials, JWT secrets, and cloud credentials shall be stored in environment variables. **DONE**
 - 4.8.1. **.env file is used. Only instructor-related credentials are stored in our repository.**

5. Privacy

- 5.1. The system shall collect only the information required for this project, including names, email address, user role, schedule information, completed courses, planned courses. **DONE**
 - 5.1.1. **80% of user data that we store are academic-related. The other 20% are personal information set up by the student user or an admin.**
- 5.2. Student schedule and academic planning data shall be used only to provide schedule, map, degree progress, and course planning functions. **DONE**
 - 5.2.1. **We collect only the relevant academic data to improve our application's usability.**

6. Compatibility

- 6.1. The application shall support the current version (149) of Google Chrome. **DONE**
 - 6.1.1. **Our application can be used on the current stable version of Google Chrome.**
- 6.2. The frontend shall operate on Windows, macOS, and Linux through a supported web browser. **DONE**
 - 6.2.1. **Our application page works on all three listed operating systems' web browsers.**
- 6.3. The frontend shall operate on other non-Windows platforms (maxOS, Linux) through a supported version of Google Chrome. **DONE**
 - 6.3.1. **Our application page works on all three listed operating systems' Google Chrome.**
- 6.4. The system shall not require users to install browser plug-in. **DONE**
 - 6.4.1. **No browser plugin is needed to use our application.**

7. Hardware and Networking Requirements

- 7.1. The application shall be deployed on an AWS Linux cloud instance. **DONE**
 - 7.1.1. **EC2 Ubuntu/AWS deployment.**
- 7.2. The cloud instance shall have sufficient resources to run the React frontend, [Node.js/Express](#) backend, Nginx, and the required application services. **ON TRACK**
 - 7.2.1. **The cloud instance has sufficient resources for ~10 concurrent users. It is unknown if the statement holds true for >20 users**
- 7.3. The application shall be accessible through a device with an internet connection and a supported web browser. **ON TRACK**
 - 7.3.1. **Per 6.1.1, our application is accessible through Windows, macOS, and Linux. And as mentioned in 3.7.1, our application is not optimized for mobile devices**
- 7.4. Remote database access shall be limited to authorized user and approved network addresses. **DONE**
 - 7.4.1. **Different sets of DB credentials are made for security reasons. Only authorized personnel can access the database.**

8. Media Content Requirements

- 8.1. Team-member photos on the About page shall use JPEG, PNG. **DONE**
 - 8.1.1. **Each photo extension is either jpeg or png.**
- 8.2. Each uploaded team-member photo shall not exceed 2 MB. **ISSUE**
 - 8.2.1. **Not all photos are under 2 MB.**
- 8.3. Images shall include meaningful alternative text for accessibility. **ON TRACK**
 - 8.3.1. **Photos used in our about page have appropriate alt text. Other images in our application are used as backdrop or background of interactive components (e.g. Campus Map) and does not require alt text.**

9. Legal, Licensing, and External Requirement

- 9.1. The system shall display a disclaimer stating that degree guidance and advanced course suggestions are planning aids and do not replace official academic advising. **ISSUE**
 - 9.1.1. **Although our system isn't intended to replace academic advisors, there is no disclaimer stating the requirement.**
- 9.2. The system shall state that graduate or advanced course suggestions require confirmation under university and department policies. **ISSUE**
 - 9.2.1. **Same as 9.1.1. There is no disclaimer for this specific statement.**
- 9.3. Course, degree, and policy information shown in the prototype shall be identified as sample data when it is not obtained from an official source. **DONE**
 - 9.3.1. **Even though many of our data (e.g. buildings, classrooms, courses, degree requirements) are based on San Francisco State University, they are meant to be treated as sample data for a class application prototype.**

10. Organizational and Development Constraints

- 10.1. The frontend shall be developed using JavaScript and React. **DONE**
 - 10.1.1. **The frontend is developed using JavaScript and React.**
- 10.2. The backend shall be developed using JavaScript, Node.js, and Express.js. **DONE**
 - 10.2.1. **The backend is developed using JavaScript, Node.js, and Express.js.**
- 10.3. The system shall use PostgreSQL as its relational database management system. **DONE**
 - 10.3.1. **The system uses PostgreSQL as its relational database management system.**

11. Coding Standards

- 11.1. Variables and functions shall be named and formatted according to the language's conventional syntax. **ON TRACK**
 - 11.1.1. **Most of our pages are formatted according to the language's syntax. Some variables may not follow the convention despite other parts of the same page do.**
- 11.2. API keys, and other private information shall not be stored inside a public folder. **DONE**
 - 11.2.1. **All private information are not stored inside the repository (minus instructor access information)**
- 11.3. All code must need approval from at least one team member before being pushed to the main branch **ON TRACK**
 - 11.3.1. **Most additions are reviewed before pushed onto the main branch. Most unique features have their own branch that requires approval from another team member to be merged.**

Team Member	Milestone 4 Responsibility
Wen Chien Yen	team coordination, usability testing, final submission, team contributions
Oscar Wang	frontend/UI polish and page consistency
Banghao Yuan	backend/API stability and remaining backend support
Sean Wang	database/integration and feature correctness
Enoch Lin	GitHub workflow, PR screenshots, code review evidence, external review
Kashif Zada	QA test plan, security self-check, validation and cross-browser testing

Team Contributions

Team lead assigned contribution scores. All team members have submitted confidential feedback via the instructor provided Google form.

Team Member	Wen Chien Yen	Oscar Wang	Banghao Yuan	Sean Wang	Enoch Lin	Kashif Zada
Contribution Score	9	9	10	10	9	9

Contribution Details

Wen-Chien Yen

I contributed to Milestone 4 as the Team Lead and Scrum Master. My main work focused on team coordination, testing planning, prototype verification, final submission support, and team contribution tracking. I reviewed the Milestone 4 README requirements and coordinated the team split between the Engineering Group and the Testing/Documentation Group. I also assigned section ownership for M4v1, helped define the five usability test functions, reviewed the prototype testing flow, and prepared communication for instructor submission and follow-up. I also helped organize the usability testing direction for the team. I confirmed that login and registration should be excluded from the usability test, helped recommend student-facing test tasks such as Course Search, Cart, Schedule, Notifications, and Degree Progress, and prepared the Team Contributions section based on M4 work history and assigned responsibilities. This helped the team keep the Milestone 4 documentation aligned with the README requirements. In addition, I verified the deployed prototype before submission. I confirmed that the public EC2 URL remained accessible, that the backend service stayed running through PM2, and that the prototype testing workflow matched both the credentials README and the main README. This helped ensure that the deployed beta prototype was ready for instructor testing and grading.

Oscar Wang

Oscar Wang contributed to Milestone 4 through frontend/UI polish and page consistency. His documented work includes updating the navbar so clicking the logo and

site name consistently sends users to the homepage, while the dedicated portal link remains available through the profile button. He also made grammar fixes to the homepage and fixed a sizing issue with the student notifications button. Oscar also worked on the frontend branch by making small changes and setting the foundation for a new profile icon for logged-in students. These changes supported the beta prototype requirement by making the interface more polished, consistent, and easier to navigate for student users. Oscar's work helped improve the overall user experience and visual quality of the application. Since Milestone 4 focuses on beta-level stability and usability, his frontend polish supported the team's goal of making the system more presentable for usability testing, grading, and the final demo.

Banghao Yuan

Banghao Yuan contributed heavily to Milestone 4 through backend/API stability, admin feature support, QA test setup, code review fixes, and frontend responsiveness improvements. His documented backend work includes adding a welcome message display for new signup users and adding decision logic so users registering with a BBedu email domain could skip the normal invite process for usability testing. This helped reduce testing friction and made it easier for outside testers to access the prototype. Banghao also enhanced admin course and degree requirement management. His work included adding course pagination, bulk import, Major Elective and GE Area support, improved Degree Requirement pages, and automatic Class Section date alignment with semester dates. These changes strengthened admin-side functionality and improved the beta prototype's completeness. He also contributed to testing and code review. He created usability test tasks, added QA test sections for reliability, security, compatibility, robustness, and performance, checked the repository, fixed issues pointed out by reviewers, and added the code review process. On 7/27/2026, he also improved mobile responsiveness by changing page sections to single-column layouts on mobile screens and preventing images from overflowing the viewport. His work strongly supported both the Engineering Group and Testing/Documentation Group requirements for Milestone 4.

Sean Wang

Sean Wang contributed heavily to Milestone 4 through documentation, usability testing, product summary refinement, security documentation, non-functional requirement review, and database/integration support. His documented work includes finalizing the list of usability test tasks, writing the Test Setup and User Profile section, creating the Google Form for usability testing, and adding survey result analysis based on received responses. Sean also completed several major M4 documentation sections. He set up the Usability Test Plan template, completed the Security Self Check section with code images, completed the AI Feature Testing and Ethics Review explanation, finished the

Non Functional Requirements Status section, and completed the Product Summary section by refining the functional requirements and rewriting the list of unique features. Sean's contribution was especially important because Milestone 4 requires formal test planning, analysis, security review, and status tracking. His work helped connect the product's functional scope with measurable usability and QA evidence. His documentation work also helped make the project look more complete and aligned with the beta prototype expectations.

Enoch Lin

Enoch Lin contributed to Milestone 4 through GitHub workflow support, CI integration, and localization documentation. His documented work includes adding a CI workflow that runs frontend lint/build checks and backend tests automatically for every pull request. This directly supports the Milestone 4 code review and engineering discipline requirement by making the team's pull request process more reliable. Enoch's CI workflow helped the team catch build and test issues before merging code into the main branch. This is important because Milestone 4 requires the default branch to remain stable for grading, and automated checks provide stronger evidence that the team is following a structured development process. Enoch also added the Localization Testing documentation on 7/27/2026. This supported one of the required M4 documentation sections and helped the team address how the application behaves with different language settings, longer translated text, and layout direction concerns.

Kashif Zada

Kashif Zada contributed to Milestone 4 through external code review, QA/security support, and validation-related testing responsibilities. His documented work includes completing an external code review of Team 1, TerrorByte-01. In that review, he identified a duplicated income/expense calculation bug across multiple services and noted that several services were missing a null-amount filter that existed in FinanceService. He also added code screenshots as evidence. This external code review contribution directly supports the Milestone 4 Code Review requirement, which asks teams to perform an external review for another team and include evidence. Kashif's work gave the team concrete review material to include in the M4 document. Kashif's assigned M4 responsibility also included QA test plan support, security self-check support, validation, and cross-browser testing. His contribution strengthened the testing/documentation side of the milestone by helping the team show evidence of review, testing awareness, and code quality evaluation.